Section A – Concept Application

Question 1

Question: Which strategy would you choose for the missing values in 'delivery_time_mins' (dropping vs mean/median imputation)? Also describe the Pandas steps to fix the 'customer_rating' column type.

Answer: I would use imputation, not dropping. 14% missing means about 7,000 of the 50,000 rows would be lost, which wastes data and can bias the result. Delivery times are usually right-skewed (a few very late orders), so the mean gets pulled up by them. The median is not affected by these extreme values, so I would fill with the median. I can confirm the skew using df['delivery_time_mins'].skew() or a histogram.

Steps to fix customer_rating:

1. Check the current type: df['customer_rating'].dtype or df.info()
2. Convert to numbers: df['customer_rating'] = pd.to_numeric(df['customer_rating'], errors='coerce') (strings like '4.5' become 4.5, bad text becomes NaN)
3. Set the type: df['customer_rating'] = df['customer_rating'].astype('float64')
4. Check new NaN values with df['customer_rating'].isnull().sum() and fill them (median/mean).
5. Confirm using df.dtypes.

Question 2

Question: How do NumPy boolean indexing and np.mean / np.std identify surge days without a loop? What is a boolean mask and why is it faster?

Answer: np.mean() and np.std() give the mean and standard deviation of the order array. Then a comparison like orders > mean + 1.5 * std is applied to the whole array at once and gives a boolean mask (an array of True/False with the same size, True for surge days). Using this mask as an index gives only the surge days, e.g. revenue[mask].sum() gives the total revenue of surge days.

```
mean = np.mean(orders)
std = np.std(orders)
mask = orders > mean + 1.5 * std
surge_revenue = revenue[mask].sum()
```

A boolean mask is an array of True/False values used to select elements. It is faster than a loop because element-wise operations run in compiled C code on a continuous block of memory (vectorisation), while a Python loop checks one element at a time with interpreter overhead.

Question 3

Question: Identify univariate / bivariate / multivariate analysis for each business question, name one Seaborn chart, and explain why it is better than a line plot.

- (a) Delivery time vs customer rating – Bivariate (two numeric variables). Chart: sns.scatterplot() (or regplot). A line plot joins points in order, which suggests a trend over time or sequence. Here every point is a separate order and has no order, so a scatter plot shows the correlation, spread and outliers properly.
- (b) Average order value across restaurant categories – Bivariate (one categorical + one numeric variable). Chart: sns.boxplot() (violin plot also works). Categories have no natural order, so a line plot joining them is meaningless and shows only one value. A boxplot shows the median, spread and outliers of order value for each category.

Question 4

Question: Which SQL JOIN is needed to include restaurants with zero orders? What happens with INNER JOIN?

Answer: A LEFT JOIN with restaurants as the left table must be used. LEFT JOIN keeps all rows of the left table, and when there is no matching order the order columns become NULL. COALESCE changes the NULL revenue to 0.

```
SELECT r.name, COALESCE(SUM(o.order_value), 0) AS total_revenue
FROM restaurants r
LEFT JOIN orders o ON r.restaurant id = o.restaurant id
GROUP BY r.restaurant_id, r.name;
```

If INNER JOIN is used, only rows that match in both tables are returned. Restaurants with no orders have no match, so they are removed from the report and the report is incomplete.

Question 5

Question: Which two window functions are used for ranking and for the previous row's value? Why are window functions with PARTITION BY better than GROUP BY with self-join or correlated subquery?

Answer: RANK() (or DENSE_RANK()) is used for ranking and LAG() is used to get the previous row's value.

```
WITH agent_counts AS (
    SELECT city, agent_id, COUNT(*) AS deliveries
    FROM deliveries
    WHERE status = 'Completed'
      AND strftime('%Y-%m', delivery_date) = strftime('%Y-%m', 'now')
    GROUP BY city, agent id
)
SELECT city, agent_id, deliveries,
       RANK() OVER (PARTITION BY city ORDER BY deliveries DESC) AS city_rank,
       LAG(deliveries) OVER (PARTITION BY city ORDER BY deliveries DESC) AS prev_agent_deliveries
FROM agent_counts;
```

PARTITION BY city restarts the rank and the LAG for every city, so the result is correct in a single pass. Ties are handled by RANK, and the top agent of each city automatically gets NULL for the previous value. A self-join or correlated subquery runs again for every row, so it is slower on big data, long, and difficult to read and change. Window functions are shorter and easier to maintain.

Question 6

Question: Compare a CTE (WITH clause) vs nested subquery for the two-step requirement (readability, reuse, debugging). When would you prefer the nested subquery?

```
WITH top_categories AS (
    SELECT r.category
    FROM orders o JOIN restaurants r ON o.restaurant_id = r.restaurant_id
    GROUP BY r.category
    ORDER BY COUNT(*) DESC
    LIMIT 5
)
SELECT o.*
FROM orders o JOIN restaurants r ON o.restaurant_id = r.restaurant_id
WHERE r.category IN (SELECT category FROM top_categories);
```

- Readability: CTE reads top to bottom and the step has a name (top_categories). A deeply nested subquery has to be read from inside to outside and is confusing.
- Reuse: a CTE can be used many times in the same query. A nested subquery must be copied again every time.
- Debugging: the CTE can be tested alone with SELECT * FROM top_categories. A nested subquery must be cut out of the main query first.

I would prefer a nested subquery when it is small, simple and used only once (for example one level WHERE x IN (SELECT ...)), because a CTE would only add extra lines. It is also useful if the database version does not support WITH.

Section B – Practical Coding Tasks

Task 1: Daily Order Analysis with NumPy

Code (task1.py):

```
import numpy as np
```

```
np.random.seed(42)
orders = np.random.randint(100, 801, 30)
revenue = np.round(np.random.uniform(5000, 50000, 30), 2)

print("Total orders:", orders.sum())
print("Total revenue:", round(revenue.sum(), 2))
mean = orders.mean()
std = orders.std()
print("Mean daily orders:", round(mean, 2))
print("Std dev of daily orders:", round(std, 2))
print("Day with highest revenue (index):", revenue.argmax(), "-> Revenue:", revenue.max())

# boolean indexing
mask = orders > mean + std
peak_days = orders[mask]
print("Peak days count:", mask.sum())
print("Peak days orders:", peak_days)
print("Revenue on peak days:", round(revenue[mask].sum(), 2))

# weekly blocks
weekly = orders.reshape(5, 6)
print("Weekly totals:", weekly.sum(axis=1))
```

Output:

```
Total orders: 12279
Total revenue: 743362.92
Mean daily orders: 409.3
Std dev of daily orders: 195.9
Day with highest revenue (index): 10 -> Revenue: 49245.39
Peak days count: 4
Peak days orders: [800 714 763 761]
Revenue on peak days: 127639.11
Weekly totals: [2284 2365 2409 3201 2020]
```

Task 2: Food Delivery Orders Data Cleaning Pipeline

Code (task2.py):

```
import pandas as pd
import numpy as np

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

# ---------- create dataset ----------
rows = [
    # order_id, customer, restaurant, category, delivery_time, order_value, rating
    (101, "Amit",   "Spice Hub",   "Indian",    30, 450,  4.5),
    (102, "Riya",   "Pizza Point", "Italian",   25, 600,  4.2),
    (103, "Karan",  "Burger Barn", "Fast Food", 20, 250,  3.9),
    (104, "Neha",   "Spice Hub",   "Indian",    35, 480,  4.6),
    (105, "Raj",    "Sushi Zen",   "Japanese",  40, 1200, 4.8),
    (106, "Pooja",  "Pizza Point", "Italian",   28, 650,  4.0),
    (107, "Vikram", "Burger Barn", "Fast Food", 22, 300,  3.7),
    (108, "Sneha",  "Sweet Tooth", "Dessert",   18, 220,  4.3),
    (109, "Arjun",  "Spice Hub",   "Indian",    33, 500,  4.4),
    (110, "Meera",  "Sushi Zen",   "Japanese",  45, 1350, 4.9),
    (111, "Rohan",  "Pizza Point", "Italian",   27, 580,  4.1),
    (112, "Anita",  "Sweet Tooth", "Dessert",   19, 260,  4.2),
    (113, "Dev",    "Burger Barn", "Fast Food", 21, 280,  3.8),
    (114, "Isha",   "Spice Hub",   "Indian",    31, 470,  4.5),
    (115, "Manish", "Sushi Zen",   "Japanese",  42, 1250, 4.7),
    (116, "Kavya",  "Pizza Point", "Italian",   26, 620,  4.0),
    (117, "Yash",   "Sweet Tooth", "Dessert",   17, 240,  4.4),
    (118, "Tina",   "Burger Barn", "Fast Food", 23, 270,  3.6),
    (119, "Harsh",  "Spice Hub",   "Indian",    34, 520,  4.3),
    (120, "Nidhi",  "Sushi Zen",   "Japanese",  44, 5200, 4.8),
]
cols = ["order_id", "customer_name", "restaurant_name", "category",
        "delivery_time_mins", "order_value", "rating"]
df = pd.DataFrame(rows, columns=cols)

# introduce missing values (5)
df.loc[2, "delivery_time_mins"] = np.nan
df.loc[6, "order_value"] = np.nan
```

```
df.loc[9, "rating"] = np.nan
df.loc[11, "category"] = np.nan
df.loc[15, "rating"] = np.nan
# add 3 duplicate rows (exact copies of rows 0, 1 and 4)
df = pd.concat([df, df.iloc[[0, 1, 4]]], ignore_index=True)


print("Raw data shape:", df.shape)
print("\nNull count BEFORE filling:\n", df.isnull().sum())


# ---------- missing values ----------
df["delivery_time_mins"] = df["delivery_time_mins"].fillna(df["delivery_time_mins"].median())
df["order_value"] = df["order_value"].fillna(df["order_value"].median())
df["rating"] = df["rating"].fillna(round(df["rating"].mean(), 1))
df["category"] = df["category"].fillna(df["category"].mode()[0])
print("\nNull count AFTER filling:\n", df.isnull().sum())


# ---------- duplicates ----------
print("\nShape before drop_duplicates:", df.shape)
df = df.drop_duplicates()
print("Shape after drop_duplicates:", df.shape)


# ---------- outliers (IQR) ----------
Q1 = df["order_value"].quantile(0.25)
Q3 = df["order_value"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
print("\nQ1:", Q1, "Q3:", Q3, "IQR:", IQR)
print("Lower fence:", lower, "Upper fence:", upper)


outliers = df[(df["order_value"] < lower) | (df["order_value"] > upper)]
print("\nOutlier rows:\n", outliers)


df["order_value"] = df["order_value"].clip(upper=upper)
print("\nMax order_value after clipping:", df["order_value"].max())
print("\nCleaned data (first 5 rows):\n", df.head())


df.to_csv("cleaned_orders.csv", index=False)
```

Output:

```
Raw data shape: (23, 7)


Null count BEFORE filling:
 order_id              0
customer_name         0
restaurant_name       0
category              1
delivery_time_mins    1
order_value           1
rating                2
dtype: int64


Null count AFTER filling:
 order_id              0
customer_name         0
restaurant_name       0
category              0
delivery_time_mins    0
order_value           0
rating                0
dtype: int64


Shape before drop_duplicates: (23, 7)
Shape after drop_duplicates: (20, 7)


Q1: 277.5 Q3: 627.5 IQR: 350.0
Lower fence: -247.5 Upper fence: 1152.5


Outlier rows:
     order_id customer_name restaurant_name  category  delivery_time_mins  order_value  rating
4        105           Raj       Sushi Zen  Japanese                40.0       1200.0     4.8
9        110         Meera       Sushi Zen  Japanese                45.0       1350.0     4.3
14       115        Manish       Sushi Zen  Japanese                42.0       1250.0     4.7
19       120         Nidhi       Sushi Zen  Japanese                44.0       5200.0     4.8
```

```
Max order_value after clipping: 1152.5

Cleaned data (first 5 rows):
    order_id customer_name restaurant_name   category  delivery_time_mins  order_value  rating
0       101          Amit       Spice Hub     Indian                30.0        450.0     4.5
1       102          Riya     Pizza Point    Italian                25.0        600.0     4.2
2       103         Karan     Burger Barn  Fast Food                29.0        250.0     3.9
3       104          Neha       Spice Hub     Indian                35.0        480.0     4.6
4       105           Raj       Sushi Zen   Japanese                40.0       1152.5     4.8
```

Task 3: Food Delivery Visualisation Dashboard

Code (task3.py):

```
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns

# cleaned dataframe from Task 2
df = pd.read_csv("cleaned_orders.csv")

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Plot 1 - countplot
sns.countplot(data=df, x="category", ax=axes[0, 0], palette="Set2")
axes[0, 0].set_title("Order Count per Category")
axes[0, 0].set_xlabel("Category")
axes[0, 0].set_ylabel("Number of Orders")

# Plot 2 - boxplot
sns.boxplot(data=df, x="category", y="delivery_time_mins", ax=axes[0, 1], palette="Set3")
axes[0, 1].set title("Delivery Time Distribution by Category")
axes[0, 1].set_xlabel("Category")
axes[0, 1].set_ylabel("Delivery Time (mins)")

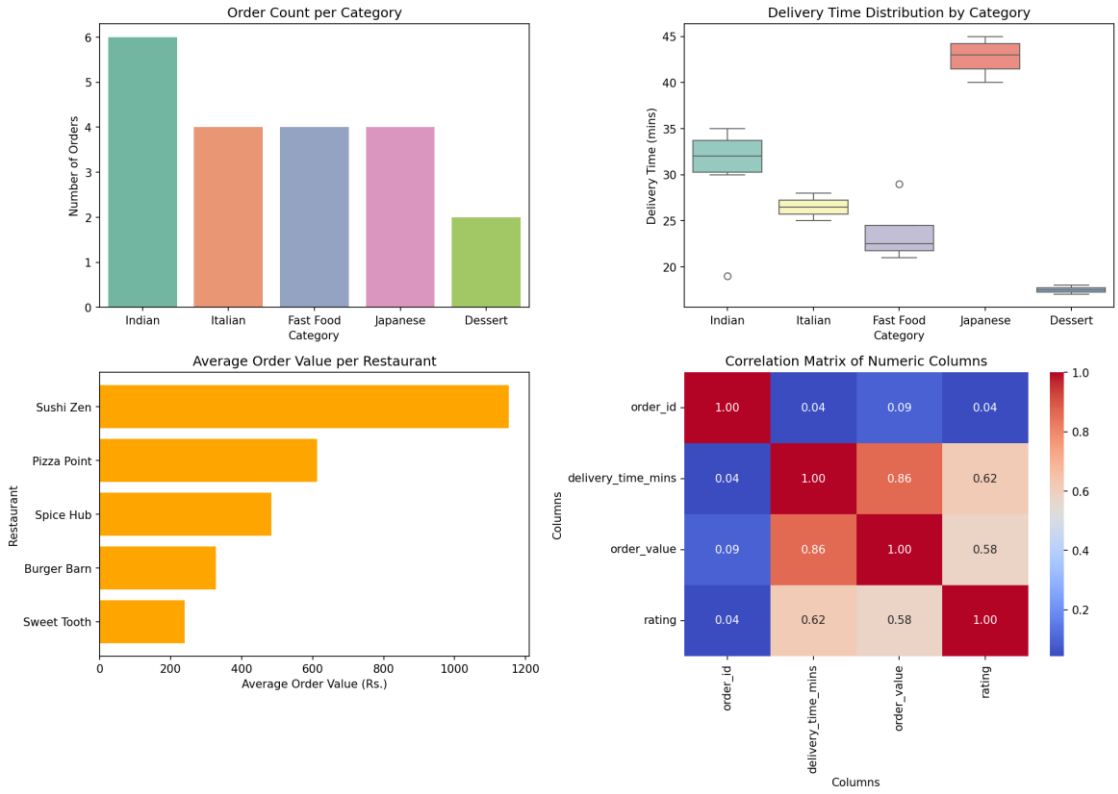
# Plot 3 - horizontal bar chart
avg_val = df.groupby("restaurant_name")["order_value"].mean().sort_values()
axes[1, 0].barh(avg_val.index, avg_val.values, color="orange")
axes[1, 0].set_title("Average Order Value per Restaurant")
axes[1, 0].set_xlabel("Average Order Value (Rs.)")
axes[1, 0].set_ylabel("Restaurant")

# Plot 4 - heatmap
corr = df[["order_id", "delivery_time_mins", "order_value", "rating"]].corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f", ax=axes[1, 1])
axes[1, 1].set_title("Correlation Matrix of Numeric Columns")
axes[1, 1].set_xlabel("Columns")
axes[1, 1].set_ylabel("Columns")

plt.tight_layout()
plt.savefig("food_delivery_dashboard.png", dpi=150, bbox_inches="tight")
print("Dashboard saved as food_delivery_dashboard.png")
```

Output:

```
Dashboard saved as food_delivery_dashboard.png
```



Task 4: SQL + Python Restaurant Performance Report

Code (task4.py):

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("food_delivery.db")
cur = conn.cursor()

cur.execute("DROP TABLE IF EXISTS orders")
cur.execute("DROP TABLE IF EXISTS restaurants")
cur.execute("""CREATE TABLE restaurants (
    restaurant_id INTEGER PRIMARY KEY, name TEXT, category TEXT, city TEXT)""")
cur.execute("""CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY, restaurant_id INTEGER, order_value REAL,
    delivery_time_mins REAL, rating REAL)""")

restaurants = [
    (1, "Spice Hub", "Indian", "Ahmedabad"),
    (2, "Pizza Point", "Italian", "Mumbai"),
    (3, "Burger Barn", "Fast Food", "Delhi"),
    (4, "Sushi Zen", "Japanese", "Mumbai"),
    (5, "Sweet Tooth", "Dessert", "Ahmedabad"),
    (6, "Taco Town", "Mexican", "Pune"),
    (7, "Dragon Wok", "Chinese", "Delhi"),
    (8, "Green Bowl", "Healthy", "Pune"),
]
orders = [
    (1, 1, 450, 30, 4.5), (2, 1, 520, 33, 4.4), (3, 1, 480, 35, 4.6),
    (4, 2, 600, 25, 4.2), (5, 2, 650, 28, 4.0), (6, 2, 580, 27, 4.1),
    (7, 3, 250, 20, 3.9), (8, 3, 300, 22, 3.7), (9, 3, 280, 21, 3.8),
    (10, 4, 1200, 40, 4.8), (11, 4, 1350, 45, 4.9), (12, 4, 1250, 42, 4.7),
    (13, 5, 220, 18, 4.3), (14, 5, 260, 19, 4.2), (15, 5, 240, 17, 4.4),
    (16, 6, 350, 26, 4.0), (17, 6, 400, 29, 4.1),
    (18, 7, 700, 32, 4.4), (19, 7, 750, 31, 4.3), (20, 7, 680, 34, 4.5),
    (21, 8, 420, 24, 4.6), (22, 8, 460, 23, 4.7),
    (23, 1, 500, 31, 4.5), (24, 4, 1300, 44, 4.8), (25, 7, 720, 33, 4.2),
]
cur.executemany("INSERT INTO restaurants VALUES (?,?,?,?)", restaurants)
cur.executemany("INSERT INTO orders VALUES (?,?,?,?,?)", orders)
conn.commit()

query = """
```

```
SELECT r.name,
       r.category,
       COUNT(o.order_id)          AS total_orders,
       SUM(o.order_value)         AS total_revenue,
       ROUND(AVG(o.rating), 2)    AS avg_rating
FROM restaurants r
JOIN orders o ON r.restaurant_id = o.restaurant_id
GROUP BY r.restaurant_id, r.name, r.category
"""
df = pd.read_sql_query(query, conn)
conn.close()

df["revenue_rank"] = df["total_revenue"].rank(ascending=False, method="min").astype(int)
df = df.sort_values("revenue_rank").reset_index(drop=True)

print("Full Restaurant Performance Report")
print(df.to_string(index=False))

print("\nTop 5 Restaurants by Total Revenue")
print(df.head(5).to_string(index=False))

df.to_csv("restaurant_performance_report.csv", index=False)
print("\nReport exported to restaurant_performance_report.csv")
```

Output:

```
Full Restaurant Performance Report
       name  category  total_orders  total_revenue  avg_rating  revenue_rank
  Sushi Zen  Japanese             4         5100.0        4.80             1
 Dragon Wok   Chinese             4         2850.0        4.35             2
  Spice Hub    Indian             4         1950.0        4.50             3
Pizza Point   Italian             3         1830.0        4.10             4
 Green Bowl   Healthy             2          880.0        4.65             5
Burger Barn Fast Food             3          830.0        3.80             6
  Taco Town   Mexican             2          750.0        4.05             7
Sweet Tooth   Dessert             3          720.0        4.30             8

Top 5 Restaurants by Total Revenue
       name category  total_orders  total_revenue  avg_rating  revenue_rank
  Sushi Zen Japanese             4         5100.0        4.80             1
 Dragon Wok  Chinese             4         2850.0        4.35             2
  Spice Hub   Indian             4         1950.0        4.50             3
Pizza Point  Italian             3         1830.0        4.10             4
 Green Bowl  Healthy             2          880.0        4.65             5

Report exported to restaurant_performance_report.csv
```

Section C – Mini Capstone Project

Food Delivery Analytics Console Application

Code (capstone.py):

```
import sqlite3
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from tabulate import tabulate


DB = "food delivery.db"
clean_df = None      # cleaned dataframe (set in option 1)
sql_results = {}     # sql results (set in option 2)



def simulate_data():
    """Create a simulated food delivery DataFrame with quality issues."""
    np.random.seed(1)
    names = ["Spice Hub", "Pizza Point", "Burger Barn", "Sushi Zen", "Sweet Tooth", "Taco Town"]
    cat = {"Spice Hub": "Indian", "Pizza Point": "Italian", "Burger Barn": "Fast Food",
           "Sushi Zen": "Japanese", "Sweet Tooth": "Dessert", "Taco Town": "Mexican"}
    city = {"Spice Hub": "Ahmedabad", "Pizza Point": "Mumbai", "Burger Barn": "Delhi",
            "Sushi Zen": "Mumbai", "Sweet Tooth": "Ahmedabad", "Taco Town": "Pune"}
    n = 30
    rest = np.random.choice(names, n)
```

```
    df = pd.DataFrame({
        "order_id": range(1, n + 1),
        "restaurant_name": rest,
        "category": [cat[r] for r in rest],
        "city": [city[r] for r in rest],
        "delivery_time_mins": np.random.randint(15, 50, n).astype(float),
        "order_value": np.random.randint(200, 900, n).astype(float),
        "rating": np.round(np.random.uniform(3.5, 5, n), 1),
    })
    df.loc[3, "order_value"] = 6000          # outlier
    df.loc[5, "delivery_time_mins"] = np.nan  # missing values
    df.loc[8, "order_value"] = np.nan
    df.loc[12, "rating"] = np.nan
    df.loc[15, "category"] = np.nan
    df = pd.concat([df, df.iloc[[0, 1]]], ignore_index=True)  # duplicates
    return df



def quality_summary(df, title):
    print(f"--- {title} ---")
    print("Rows:", df.shape[0], "| Columns:", df.shape[1])
    print("Missing values:", int(df.isnull().sum().sum()))
    print("Duplicate rows:", int(df.duplicated().sum()))
    print("Max order_value:", df["order_value"].max())



def load_and_clean():
    global clean_df
    df = simulate_data()
    quality_summary(df, "BEFORE cleaning")

    df["delivery_time_mins"] = df["delivery_time_mins"].fillna(df["delivery_time_mins"].median())
    df["order_value"] = df["order_value"].fillna(df["order_value"].median())
    df["rating"] = df["rating"].fillna(round(df["rating"].mean(), 1))
    df["category"] = df["category"].fillna(df["category"].mode()[0])
    df = df.drop_duplicates()

    q1, q3 = df["order_value"].quantile([0.25, 0.75])
    upper = q3 + 1.5 * (q3 - q1)
    print("Outliers capped at:", upper)
    df["order_value"] = df["order_value"].clip(upper=upper)

    clean_df = df
    quality_summary(df, "AFTER cleaning")



def setup_database():
    conn = sqlite3.connect(DB)
    cur = conn.cursor()
    cur.execute("DROP TABLE IF EXISTS orders")
    cur.execute("DROP TABLE IF EXISTS restaurants")
    cur.execute("""CREATE TABLE restaurants (
        restaurant_id INTEGER PRIMARY KEY, name TEXT, category TEXT, city TEXT)""")
    cur.execute("""CREATE TABLE orders (
        order_id INTEGER PRIMARY KEY, restaurant_id INTEGER, order_value REAL,
        delivery_time_mins REAL, rating REAL)""")
    rests = [(1, "Spice Hub", "Indian", "Ahmedabad"), (2, "Pizza Point", "Italian", "Mumbai"),
             (3, "Burger Barn", "Fast Food", "Delhi"), (4, "Sushi Zen", "Japanese", "Mumbai"),
             (5, "Sweet Tooth", "Dessert", "Ahmedabad"), (6, "Taco Town", "Mexican", "Pune")]
    ords = [(1, 1, 450, 30, 4.5), (2, 1, 520, 33, 4.4), (3, 2, 600, 25, 4.2), (4, 2, 650, 28,
4.0),
            (5, 3, 250, 20, 3.9), (6, 3, 300, 22, 3.7), (7, 4, 1200, 40, 4.8), (8, 4, 1350, 45,
4.9),
            (9, 5, 220, 18, 4.3), (10, 5, 260, 19, 4.2), (11, 6, 350, 26, 4.0)]
    cur.executemany("INSERT INTO restaurants VALUES (?,?,?,?)", rests)
    cur.executemany("INSERT INTO orders VALUES (?,?,?,?,?)", ords)
    conn.commit()
    conn.close()



def run_sql():
    global sql_results
    setup_database()
    conn = sqlite3.connect(DB)
```

```
    q1 = """SELECT category, COUNT(*) AS total_orders, SUM(order_value) AS total_revenue,
                   ROUND(AVG(rating), 2) AS avg_rating
            FROM restaurants r JOIN orders o ON r.restaurant_id = o.restaurant_id
            GROUP BY category ORDER BY total_revenue DESC"""
    q2 = """SELECT o.order_id, r.name AS restaurant, r.city, o.order_value, o.rating
            FROM orders o JOIN restaurants r ON o.restaurant_id = r.restaurant_id
            ORDER BY o.order_value DESC LIMIT 5"""
    df1 = pd.read_sql_query(q1, conn)
    df2 = pd.read_sql_query(q2, conn)
    conn.close()

    sql results = {"Category Summary": df1, "Top Orders": df2}
    print("\nQuery 1: GROUP BY with COUNT, SUM, AVG")
    print(tabulate(df1, headers="keys", tablefmt="psql", showindex=False))
    print("\nQuery 2: JOIN restaurants and orders (Top 5 orders)")
    print(tabulate(df2, headers="keys", tablefmt="psql", showindex=False))



def view_charts():
    if clean_df is None:
        print("Please run option 1 first (Load & Clean Data).")
        return
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    sns.countplot(data=clean_df, x="category", ax=axes[0], hue="category", legend=False)
    axes[0].set_title("Orders by Category")
    axes[0].set_xlabel("Category")
    axes[0].set_ylabel("Number of Orders")
    axes[0].tick_params(axis="x", rotation=30)

    sns.barplot(data=clean_df, x="city", y="delivery_time_mins", ax=axes[1],
                hue="city", legend=False, errorbar=None)
    axes[1].set_title("Average Delivery Time by City")
    axes[1].set_xlabel("City")
    axes[1].set_ylabel("Avg Delivery Time (mins)")
    plt.tight_layout()
    plt.savefig("capstone_charts.png", dpi=120)
    plt.show()



def export_report():
    if clean_df is None or not sql_results:
        print("Please run option 1 and option 2 first.")
        return
    path = "food_delivery_report.xlsx"
    with pd.ExcelWriter(path) as writer:
        clean_df.to_excel(writer, sheet_name="Cleaned_Data", index=False)
        sql_results["Category_Summary"].to_excel(writer, sheet_name="SQL_Category_Summary",
index=False)
        sql_results["Top_Orders"].to_excel(writer, sheet_name="SQL_Top_Orders", index=False)
    print("Report exported successfully to:", path)



def main():
    while True:
        print("\n===== FOOD DELIVERY ANALYTICS =====")
        print("1. Load & Clean Data")
        print("2. Run SQL Analysis")
        print("3. View Charts")
        print("4. Export Report")
        print("0. Exit")
        choice = input("Enter your choice: ").strip()
        if choice == "1":
            load_and_clean()
        elif choice == "2":
            run sql()
        elif choice == "3":
            view_charts()
        elif choice == "4":
            export_report()
        elif choice == "0":
            print("Goodbye!")
            break
        else:
            print("Invalid choice, please enter 0-4.")
```

```
if __name__ == "__main__":
    main()
```

Output (options 1, 2, 4, 0 run in order):

```
===== FOOD DELIVERY ANALYTICS =====
1. Load & Clean Data
2. Run SQL Analysis
3. View Charts
4. Export Report
0. Exit
Enter your choice: --- BEFORE cleaning ---
Rows: 32 | Columns: 7
Missing values: 4
Duplicate rows: 2
Max order_value: 6000.0
Outliers capped at: 1262.0
--- AFTER cleaning ---
Rows: 30 | Columns: 7
Missing values: 0
Duplicate rows: 0
Max order_value: 1262.0

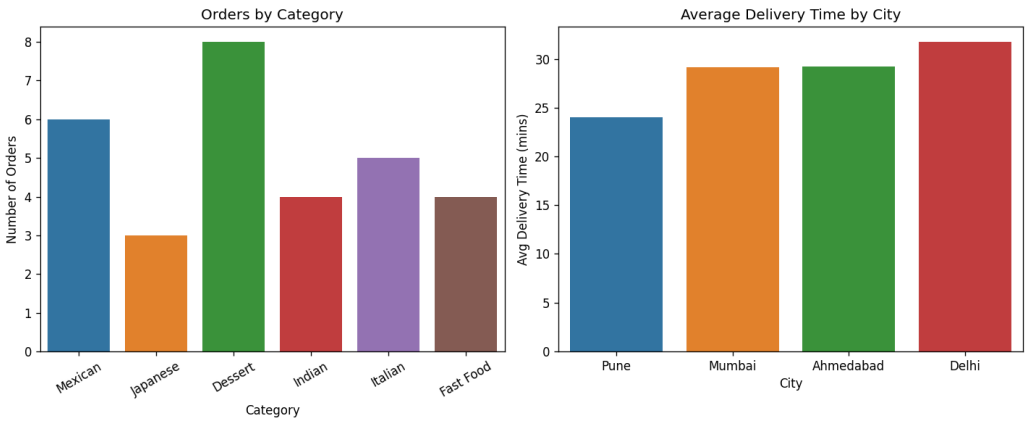
===== FOOD DELIVERY ANALYTICS =====
1. Load & Clean Data
2. Run SQL Analysis
3. View Charts
4. Export Report
0. Exit
Enter your choice:
Query 1: GROUP BY with COUNT, SUM, AVG
+------------+----------------+-----------------+--------------+
| category   |   total_orders |   total_revenue |   avg_rating |
|------------+----------------+-----------------+--------------|
| Japanese   |              2 |            2550 |         4.85 |
| Italian    |              2 |            1250 |         4.1  |
| Indian     |              2 |             970 |         4.45 |
| Fast Food  |              2 |             550 |         3.8  |
| Dessert    |              2 |             480 |         4.25 |
| Mexican    |              1 |             350 |         4    |
+------------+----------------+-----------------+--------------+

Query 2: JOIN restaurants and orders (Top 5 orders)
+------------+--------------+-----------+---------------+-----------+
|   order_id | restaurant   | city      |   order_value |   rating |
|------------+--------------+-----------+---------------+-----------|
|          8 | Sushi Zen    | Mumbai    |          1350 |      4.9 |
|          7 | Sushi Zen    | Mumbai    |          1200 |      4.8 |
|          4 | Pizza Point  | Mumbai    |           650 |      4   |
|          3 | Pizza Point  | Mumbai    |           600 |      4.2 |
|          2 | Spice Hub    | Ahmedabad |           520 |      4.4 |
+------------+--------------+-----------+---------------+-----------+

===== FOOD DELIVERY ANALYTICS =====
1. Load & Clean Data
2. Run SQL Analysis
3. View Charts
4. Export Report
0. Exit
Enter your choice: Report exported successfully to: food_delivery_report.xlsx

===== FOOD DELIVERY ANALYTICS =====
1. Load & Clean Data
2. Run SQL Analysis
3. View Charts
4. Export Report
0. Exit
Enter your choice: Goodbye!
```

Option 3 charts:



Section D – AI-Augmented Learning

1. Prompt given to the AI tool

```
Write a Python program that connects to a SQLite database containing a food delivery
orders table (columns: order_id, restaurant_name, category, order_value, rating,
delivery_time_mins) and finds the top 3 restaurants by average customer rating, ignoring
any restaurant with fewer than 5 orders. Load the SQL result into a Pandas DataFrame and
add a column with each restaurant's revenue share as a percentage of the total revenue of
the top 3 restaurants. Plot a horizontal bar chart of the top 3 restaurants by average
rating with the revenue share percentage as a label on each bar. Handle the case where the
database file does not exist or the query returns no rows by printing a clear error
message instead of crashing.
```

2. AI's original code and corrected version

AI's original code:

```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt

conn = sqlite3.connect("orders.db")

query = """
SELECT restaurant_name,
       ROUND(AVG(rating), 2) AS avg_rating,
       SUM(order_value) AS total_revenue
FROM orders
HAVING COUNT(*) >= 5
GROUP BY restaurant_name
ORDER BY avg_rating DESC
LIMIT 3
"""
df = pd.read_sql_query(query, conn)
conn.close()

df["revenue_share"] = df["total_revenue"] / df["total_revenue"].sum() * 100

fig, ax = plt.subplots()
bars = ax.barh(df["restaurant_name"], df["avg_rating"])
ax.bar_label(bars, labels=[f"{x:.1f}%" for x in df["revenue share"]])
plt.show()
```

Output of original code:

```
sqlite3.OperationalError: near "GROUP": syntax error
pandas.errors.DatabaseError: Execution failed on sql ... near "GROUP": syntax error
```

Corrected version (my code):

```
import os
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
```

```
DB_FILE = "orders.db"

# fix 1: handle missing database file
if not os.path.exists(DB_FILE):
    print("Error: database file", DB_FILE, "not found. Please check the file name/path.")
    raise SystemExit

conn = sqlite3.connect(DB_FILE)

# fix 2: GROUP BY first, then HAVING
query = """
SELECT restaurant_name,
       COUNT(*) AS total_orders,
       ROUND(AVG(rating), 2) AS avg_rating,
       SUM(order_value) AS total_revenue
FROM orders
GROUP BY restaurant_name
HAVING COUNT(*) >= 5
ORDER BY avg_rating DESC
LIMIT 3
"""
try:
    df = pd.read_sql_query(query, conn)
except Exception as e:
    print("Error while running the query:", e)
    raise SystemExit
finally:
    conn.close()

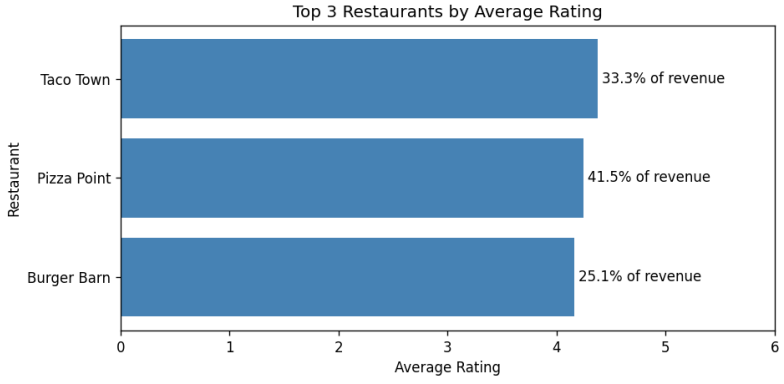
# fix 3: handle empty result
if df.empty:
    print("No restaurant has 5 or more orders. Nothing to show.")
    raise SystemExit

# fix 4: float division (*100.0) for revenue share
df["revenue_share_pct"] = (df["total_revenue"] * 100.0 / df["total_revenue"].sum()).round(1)
print(df)

fig, ax = plt.subplots(figsize=(8, 4))
bars = ax.barh(df["restaurant_name"], df["avg_rating"], color="steelblue")
ax.bar_label(bars, labels=[f"{x}% of revenue" for x in df["revenue_share_pct"]], padding=3)
# fix 5: titles and axis labels
ax.set_title("Top 3 Restaurants by Average Rating")
ax.set_xlabel("Average Rating")
ax.set_ylabel("Restaurant")
ax.set_xlim(0, 6)
ax.invert_yaxis()
plt.tight_layout()
plt.savefig("top3_chart.png", dpi=120)
plt.show()
```

Output of corrected code:

```
  restaurant_name  total_orders  avg_rating  total_revenue  revenue_share_pct
0       Taco Town             5        4.38         3677.0               33.3
1     Pizza Point             6        4.25         4583.0               41.5
2     Burger Barn             5        4.16         2775.0               25.1
```



Test when database file is missing:

```
Error: database file orders.db not found. Please check the file name/path.
```

3. Note on what I changed and why

The AI placed HAVING before GROUP BY, which gave a syntax error, so I moved HAVING after GROUP BY. It also did not check if the database file exists (sqlite3.connect silently creates an empty file) or if the DataFrame is empty before plotting, so I added os.path.exists() and df.empty checks with friendly messages. I added a chart title and axis labels, and used 100.0 in the percentage formula so it is always float division. I also added total_orders in the query so the 5-order filter can be verified.